Homework — 1.4.3 Boolean Algebra — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Total: Part A 14 + Part B 6 = 20 marks.

Part A

1. [4] Q = (A + B) · ¬C:

A	B	C	A+B	¬C	Q
0	0	0	0	1	**0**
0	0	1	0	0	**0**
0	1	0	1	1	**1**
0	1	1	1	0	**0**
1	0	0	1	1	**1**
1	0	1	1	0	**0**
1	1	0	1	1	**1**
1	1	1	1	0	**0**

Mark: A+B column correct (1); ¬C column correct (1); Q correct for rows 1–4 (1); Q correct for rows 5–8 — Q = 1 only when (A OR B) is true AND C = 0 (1).

2. [3] Simplify `¬(A · ¬B)`: - Apply De Morgan's to the outer bar: `¬(A · ¬B) = ¬A + ¬(¬B)` (break the bar, change · to +, negate each term). (1) - Apply double negation: `¬(¬B) = B`. (1) - Result: **`¬A + B`**. (1)

3. [3] Simplify `A + ¬A · B`: - This is a standard absorption/redundancy result. Using the distributive law: `A + ¬A·B = (A + ¬A) · (A + B)`. (1 — valid method/identity) - `A + ¬A = 1` (complement law), so the expression becomes `1 · (A + B)`. (1) - `1 · (A + B) = A + B` (identity law). Result: **`A + B`**. (1) *(Naming distributive + complement + identity, or quoting the redundancy law `A + ¬A·B = A + B` directly with justification, gains full marks.)*

4. [2] Half adder: **Sum = A ⊕ B**, **Carry = A · B** (1 — both correct). It cannot be used alone for the middle columns because a half adder has **only two inputs (A and B) and no carry-in**, so it cannot accept the **carry produced by the previous column** — the middle and higher columns of a multi-bit addition each need to add three bits (A, B and the carry-in), which requires a **full adder** (1). *(AO1)*

5. [2] Any two distinct differences (1 each, max 2), e.g.: - Combinational logic output depends **only on the current inputs**; sequential logic output depends on the current inputs **and on stored previous state** (memory). - Combinational logic has **no memory / no clock**; sequential logic **stores state** and is typically **clock/edge-triggered**. Examples: combinational — logic gate / half adder / full adder; sequential — **D-type flip-flop** / register / counter. *Mark: one valid difference + example pairing (1); a second distinct valid difference + example (1).*

Part B

6. [3] Symmetric encryption uses the **same key** to encrypt and decrypt, whereas asymmetric encryption uses a **pair of keys** — a **public key** to encrypt and the matching **private key** to decrypt (1 — for the key difference). Symmetric is **faster** but suffers the **key-distribution problem** (the shared key must be transmitted securely) (1). Advantage of asymmetric (any one): the public key can be **shared openly**, which solves the key-distribution problem; it also enables **digital signatures** (sign with private key, verify with public key) (1). *(AO1)*

7. [3] Round robin gives each process a **fixed time slice (quantum)** of CPU time in turn (1). If a process does not finish within its slice it is **paused and moved to the back of the queue**, and the next process runs; this cycle repeats until all processes complete (1 — it is pre-emptive). Advantage (any one): it is **fair** — every process gets regular CPU time, so **no process starves**, and it gives good responsiveness for interactive tasks (1). *(AO1)*